

Процесс запуска задач

Компания: Nexora Systems (IT-сервисы для B2B)

Документ фиксирует полный цикл задачи: планирование, реализацию, тестирование и релиз.

Этапы и контроль выполнения

1. Инициатор формирует задачу в Jira с описанием цели, ценности и критериев приемки.
2. Тимлид проводит приоритизацию и назначает исполнителя.
3. Исполнитель уточняет требования, декомпозирует задачу и обновляет оценку.
4. Разработка ведется в feature-ветке с обязательными коммитами по этапам.
5. Перед релизом проходят code review, unit-тесты и smoke-тест на стенде.
6. После успешной проверки задача переводится в статус Done и публикуется в release notes.
7. Первичный разбор новой задачи выполняется в течение 4 рабочих часов.
8. Назначение исполнителя по задаче выполняется не позднее следующего рабочего дня.
9. При блокировке задачи инициируется эскалация в рабочем чате и в Jira.
10. Для задач с внешними интеграциями составляется отдельный план тестирования.
11. Каждая задача должна иметь ссылку на исходное бизнес-требование.
12. Перед разработкой фиксируется влияние задачи на метрики продукта.
13. Если меняется схема БД, создается миграция и план отката.
14. Для критичных задач выполняется дополнительная проверка на staging.
15. Тестировщик подтверждает прохождение сценариев из acceptance criteria.
16. После релиза владелец задачи проверяет логи и ключевые метрики в течение 30 минут.
17. При неуспешном релизе выполняется rollback по инструкции дежурной команды.
18. В конце спринта формируется отчет по velocity и причинам отклонений.
19. Переоткрытие задач допускается только с указанием причины и нового плана.
20. Время выполнения задачи учитывается в разрезе разработки, теста и согласований.
21. Результат демонстрации фиксируется в комментарии к задаче.
22. Если задача влияет на клиентов, готовится шаблон внешней коммуникации.
23. Изменения в SLA фиксируются в отдельной карточке согласования.
24. Пакет задач для релиза формируется минимум за 24 часа до выкладки.
25. Все блокиеры релиза обсуждаются на Go/No-Go встрече.
26. тимлид по теме 'декомпозиция' уточняет критерии приемки с заказчиком и фиксирует результат в карточке Jira, чтобы сократить время поставки.
27. Для направления 'планирование релиза' команда выделяет зависимые задачи в отдельный блок; ограничение: не откладывать эскалацию блокиеров до конца дня, ожидаемый эффект — избежать откатов в продакшене.
28. По процессу 'процесс запуска задач' принято, что инженер поддержки готовит план отката на случай неудачного релиза и передает артефакт в журнале изменений.
29. Если возникает ситуация по теме 'тестирование', то разработчик действует в рамках SLA и соблюдает правило: не закрывать тестовый этап при незакрытых дефектах P1.
30. В рабочем цикле 'процесс запуска задач' используется практика: публикует статус релизной готовности; подтверждение выполнения хранится в протоколе Go/No-Go.

31. При изменениях в области 'декомпозиция' команда выбирает сценарий 'уточняет критерии приемки с заказчиком', чтобы добиться сократить время поставки.
32. Для предсказуемости процесса тимлид ведет коммуникацию в течение рабочего дня, ориентируясь на не откладывать эскалацию blockers до конца дня.
33. Ключевая договоренность по теме 'эскалация blockers': готовит план отката на случай неудачного релиза; после шага обновляется журнале изменений и метрика успеха.
34. инженер поддержки по теме 'тестирование' согласует тестовый периметр со смежными командами и фиксирует результат в плане тестирования, чтобы улучшить качество поставки.
35. Для направления 'выкатка' команда публикует статус релизной готовности; ограничение: не запускать релиз без подтверждения ответственного, ожидаемый эффект — ускорить синхронизацию команд.
36. По процессу 'процесс запуска задач' принято, что product owner уточняет критерии приемки с заказчиком и передает артефакт в карточке Jira.
37. Если возникает ситуация по теме 'планирование релиза', то дежурный инженер действует до утреннего стендапа и соблюдает правило: не откладывать эскалацию blockers до конца дня.
38. В рабочем цикле 'процесс запуска задач' используется практика: готовит план отката на случай неудачного релиза; подтверждение выполнения хранится в журнале изменений.
39. При изменениях в области 'тестирование' команда выбирает сценарий 'согласует тестовый периметр со смежными командами', чтобы добиться улучшить качество поставки.
40. Для предсказуемости процесса инженер поддержки ведет коммуникацию до релизного окна, ориентируясь на не запускать релиз без подтверждения ответственного.
41. Ключевая договоренность по теме 'декомпозиция': уточняет критерии приемки с заказчиком; после шага обновляется карточке Jira и метрика успеха.
42. product owner по теме 'планирование релиза' выделяет зависимые задачи в отдельный блок и фиксирует результат в release checklist, чтобы избежать откатов в продакшене.
43. Для направления 'эскалация blockers' команда готовит план отката на случай неудачного релиза; ограничение: не выпускать задачу без проверки метрик, ожидаемый эффект — снизить риск инцидентов в релизе.
44. По процессу 'процесс запуска задач' принято, что тимлид согласует тестовый периметр со смежными командами и передает артефакт в плане тестирования.
45. Если возникает ситуация по теме 'выкатка', то системный аналитик действует до конца спринта и соблюдает правило: не запускать релиз без подтверждения ответственного.
46. В рабочем цикле 'процесс запуска задач' используется практика: уточняет критерии приемки с заказчиком; подтверждение выполнения хранится в карточке Jira.
47. При изменениях в области 'планирование релиза' команда выбирает сценарий 'выделяет зависимые задачи в отдельный блок', чтобы добиться избежать откатов в продакшене.
48. Для предсказуемости процесса product owner ведет коммуникацию в течение 2 часов, ориентируясь на не выпускать задачу без проверки метрик.
49. Ключевая договоренность по теме 'тестирование': согласует тестовый периметр со смежными командами; после шага обновляется плане тестирования и метрика успеха.
50. тимлид по теме 'выкатка' публикует статус релизной готовности и фиксирует результат в протоколе Go/No-Go, чтобы ускорить синхронизацию команд.
51. Для направления 'декомпозиция' команда уточняет критерии приемки с заказчиком; ограничение: не объединять критичные изменения без smoke-теста, ожидаемый эффект — сократить время поставки.

52. По процессу 'процесс запуска задач' принято, что инженер поддержки выделяет зависимые задачи в отдельный блок и передает артефакт в release checklist.
53. Если возникает ситуация по теме 'эскалация blockers', то разработчик действует в рамках SLA и соблюдает правило: не выпускать задачу без проверки метрик.
54. В рабочем цикле 'процесс запуска задач' используется практика: согласует тестовый периметр со смежными командами; подтверждение выполнения хранится в плане тестирования.
55. При изменениях в области 'выкатка' команда выбирает сценарий 'публикует статус релизной готовности', чтобы добиться ускорить синхронизацию команд.
56. Для предсказуемости процесса тимлид ведет коммуникацию в течение рабочего дня, ориентируясь на не объединять критичные изменения без smoke-теста.
57. Ключевая договоренность по теме 'планирование релиза': выделяет зависимые задачи в отдельный блок; после шага обновляется release checklist и метрика успеха.
58. инженер поддержки по теме 'эскалация blockers' готовит план отката на случай неудачного релиза и фиксирует результат в журнале изменений, чтобы снизить риск инцидентов в релизе.
59. Для направления 'тестирование' команда согласует тестовый периметр со смежными командами; ограничение: не закрывать тестовый этап при незакрытых дефектах P1, ожидаемый эффект — улучшить качество поставки.
60. По процессу 'процесс запуска задач' принято, что product owner публикует статус релизной готовности и передает артефакт в протоколе Go/No-Go.